MiniDB · файловая мини-СУБД

Устройство, хранение и команды

Пример создаёт ./sample.sh: скрипт передаёт SQL в run.sh. Все три таблицы создаются сразу через CREATE TABLE; ADD COLUMN проверяется отдельно. INSERT добавляет 5/5/20 записей. Открытие: ./run.sh sample. Rust-модуля со схемами примера нет.

Схема данных

datasets

id INT · PK

name STRING

task STRING

size INT ≥ 0

experiments

id INT · PK

dataset_id INT · FK

model_id INT · FK

accuracy FLOAT [0,1]

status STRING

models

id INT · PK

name STRING

version STRING

experiments.dataset_id → datasets.id; experiments.model_id → models.id. Связи N:1, RESTRICT при удалении и изменении ключа родителя. Все поля примера NOT NULL; при INSERT в datasets обязательны name, task и size. DBML для dbdiagram: docs/schema.dbml.

Фиксированные записи и текстовые схемы

Файл имя.schema.sql содержит схему с CREATE TABLE, VERSION и COLUMNS_AT_VERSION; IDE подсвечивает SQL. Сегменты имя.vN.data содержат бинарные записи фиксированной длины в рамках своей версии. Все версии, включая первую, имеют явный номер. Прежние .schema/.data продолжают читаться. Layout хранит смещения. Запись содержит флаг, NULL-bitmap и поля: числа little-endian, STRING с двухбайтной длиной и UTF-8. Индекс PK связывает id с версией сегмента и смещением записи.

INSERT сохраняет FREE, затем тело с FREE, затем LIVE; между этапами — sync_data. Удаление меняет флаг на DEAD, слот можно использовать снова. Максимальный ID восстанавливается из LIVE/DEAD; checkpoint в схеме сохраняется при sync и перед утратой старого PK при reuse или смене ключа. Append не переписывает схему. Рабочего WAL нет; Пакетные UPDATE и DELETE не имеют отката при ошибке I/O. Backup копирует схемы и все сегменты после sync.

Примеры команд внутри MiniDB

```
CREATE TABLE notes (id INT PRIMARY KEY, name STRING);
INSERT INTO notes VALUES (NULL, 'Alice');
SELECT name FROM notes WHERE id = 1;
UPDATE notes SET name = 'Ann' WHERE id = 1;
DELETE FROM notes WHERE id = 1;
backup 'data/copy'
restore 'data/copy' to 'data/restored'
```

Также: проекция, шесть сравнений, AND/OR со скобками, IS NULL и INNER JOIN INT-поля левой таблицы с PK правой. Изменения сохраняются после каждой успешной команды.

MiniDB · файловая мини-СУБД

Проверка, benchmark и ограничения

Индекс: полный просмотр и точечное чтение

В RAM находится HashMap id → (версия сегмента, смещение записи). Scan и восстановление индекса читают через BufReader на 64 КиБ. Записи используют один рабочий буфер; поля заимствуют его память. at() декодирует поле, а ограничения проверяются отдельно при записи и через Table::check().

Release, 5000 записей, 1000 одинаковых детерминированных запросов; прогретый файловый кэш. Генерация, commit/fsync и открытие исключены из измерения поиска. Контрольные суммы результатов совпали: 2486500.

Полный просмотр	74.849 мс
Индекс id → смещение	0.479 мс
Отношение времени	156.33×

Это один локальный замер, не гарантия скорости на другой машине. Воспроизведение: cargo run --release --example bench -- scratch/bench 5000. Для другого числа записей нужен новый каталог.

Проверенное поведение

78 тестов: CRUD и рестарт; фильтры и NULL; PK/auto-ID, FK, RESTRICT и JOIN; backup/restore и блокировка; числа, STRING и UTF-8; консоль, sample.sh, эволюция схем и перенос старого формата. SHOW DATABASE показывает абсолютный путь активной базы.

Тесты моделируют FREE-слот с оборванным телом, неполный хвост, повреждённые флаги и UTF-8. Проверяются сохранение максимального ID после удаления и переиспользования слота, явная проверка ограничений и отказ restore при повреждении данных.

Инженерное расширение

SQL-подобный парсер, консоль и Docker с постоянным томом. Контейнер собирает и проверяет код, запускает консоль непривилегированным пользователем. Запуск: docker compose run --build --rm minidb. Инструкции запуска sample.sh внутри контейнера есть в README.

Границы реализации

Индекс требует O(числа живых записей) памяти. До 256 полей; STRING по умолчанию 64 байта, максимальная ёмкость и длина — 65535 байт. Пакет UPDATE/DELETE ограничен 16 МиБ. Нет составных PK, self JOIN, vacuum и многооператорных транзакций.

ALTER ADD COLUMN добавляет nullable-поле в конец и новый сегмент; старые строки читают его как NULL. Генератор создаёт Rust-код из схемы; потребитель компилирует его своим build.rs. Типизированная запись проверяет VERSION. Перенос UPDATE между сегментами не транзакционен. Порядок FREE → LIVE не гарантирует атомарность при повреждении сектора. Физическое отключение питания не тестировалось.

Материалы: README.md, sample.sh, docs/schema.dbml, docs/schema.svg, docs/benchmark.csv.